

3rd Feb 2026 / DAG401

⇒ Intro to Backpropagaⁿ → algo. for calculating gradients w.r.t. params of network
 ↳ chain rule from output to input layer.
 ↳ It tells each wt. contributed to the error, which we can adjust to reduce error.

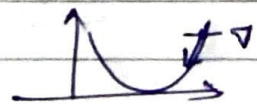
$$\frac{dL}{dw} = \frac{L(w + \Delta w) - L(w)}{\Delta w} \quad \left. \vphantom{\frac{dL}{dw}} \right\} \begin{array}{l} \text{inefficient} \\ \text{for small} \\ \text{NNs also.} \end{array}$$

- Min loss L

$$\Delta w \propto - \frac{\partial L}{\partial w}$$

move in
the direcⁿ of
steepest direcⁿ

we need this gradient vector
 ↳ steepest direcⁿ ⇒ min the error



- For every wt. w_{jk}^l & bias b_k^l , we compute

$$\frac{\partial L}{\partial w_{jk}^l} \quad \& \quad \frac{\partial L}{\partial b_k^l}$$

$L \rightarrow$ loss funcⁿ

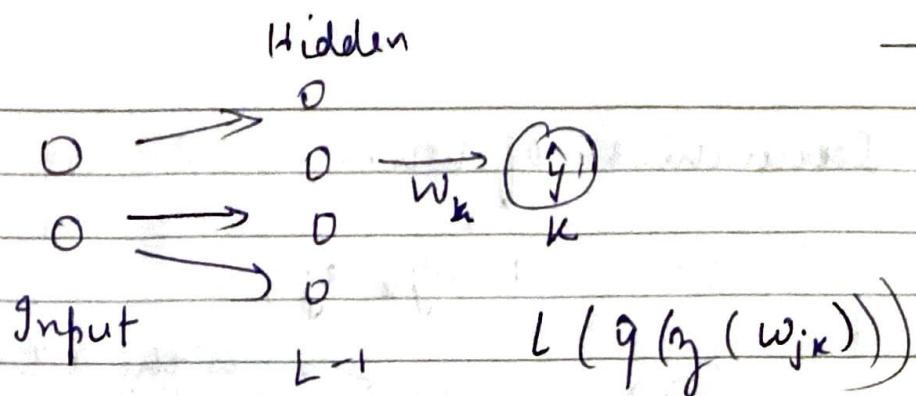
$w_{jk}^l \rightarrow$ wt. connecting neuron j in layer $l-1$ to neuron k in layer l .

$b_k^l \rightarrow$ bias for neuron k in layer l .

- As per the chain rule -

$y \rightarrow$ true, $\hat{y} \rightarrow$ predicted, $z \Rightarrow$ output (wt. sum) made

$$\boxed{\frac{\partial L}{\partial w_{jk}^l} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_{jk}^l}} \quad \underbrace{\hspace{10em}}_{\frac{\partial L}{\partial z}}$$



- ① Derivative of Loss

$$\text{BCE} \Rightarrow L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

how does error δ as predicⁿ $\hat{y}$ as.

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

$$\frac{\partial L}{\partial y} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

- ② Derivative of sigmoid

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \hat{y}(1-\hat{y})$$

- ③ Multiply,

$$\delta = \hat{y} - y$$

local error is just ^{diff. btw.} prediction & truth

$$L = (\hat{y} - y)^2 \Rightarrow \text{similarly calculate for this.}$$

- Derivatives of wts.

$$z = \dots + w_{jk} a_j \dots$$

$$\frac{\partial z}{\partial w_{jk}} = a_j \rightarrow \text{is the activat}^n \text{ of the hidden neuron that detects features}$$

$$\frac{\partial L}{\partial w_{jk}} = (\hat{y} - y) \cdot a_j$$

δ $\downarrow$ $\downarrow$ $\downarrow$
 error input

If predicⁿ $\approx$ truth, then gradient is 0.
 The learning stops. Error flows backward from output to input.

- Review

- ① Error at the output layer

$$\delta_k = \frac{\partial L}{\partial z_k} = \hat{y} - y \quad z_k \rightarrow \text{wt. sum into the output neuron}$$

- ② Error at the hidden layer

$$\delta_j = \frac{\partial L}{\partial z_j} = \underbrace{\frac{\partial L}{\partial z_k}}_{\delta_k} \cdot \underbrace{\frac{\partial z_k}{\partial a_j}}_{w_{jk}} \cdot \underbrace{\frac{\partial a_j}{\partial z_j}}_{\sigma'(z_j)}$$

$$\delta_j = \delta_k \cdot w_{jk} \cdot \sigma'(z_j)$$

- ③ update input to hidden wts.

$$\frac{\partial L}{\partial v_{ij}} = \delta_j \cdot x_i$$

error at hidden neuron j

input from feature i

$$\nabla_w L = \delta x^T$$

- General Backprop.

L layers $\longrightarrow$ layer 1

① Output Layer (layer L) $\delta^{(L)} = \hat{y} - y$

② Hidden ($l = L-1, L-2, \dots, 2$)

$$\delta^l = W^{l+1} (\delta^{l+1}) \odot \sigma'(z^{(l)})$$

$\downarrow$
element-wise multiplication

③ Wt. updates (all layers)

$$\frac{\partial L}{\partial W^l} = \delta^l (a^{l-1})^T \dots$$

$\Rightarrow$ the error flows backward through the nets., that are scaled by activation derivatives

- Few insights why the backprop. works -

1. chain rule is everything

2. we can reuse the computations

3. error attribuⁿ $\rightarrow$ wts. that contributed more to the error will get larger updates.

4. vanishing or exploding gradients